

2_세대 별 수집 원리 - GC의 실제 동작 방식

1. 쓰레기란 무엇이고, 순진한 방식의 한계

쓰레기(garbage): 실행 중인 프로그램에서, 살아있는 다른 객체의 어떤 참조로부터도 더 이상 도달할 수 없게 된 객체.

여기서 "참조로 도달할 수 있는가"를 **도달 가능성(Reachability)** 이라고 한다.

가장 순진한 방식은 이렇다. GC가 실행될 때마다 도달 가능한 모든 객체를 전부 순회하고, 순회되지 않은 나머지를 쓰레기로 처리하는 것이다.

문제는 비용이다.

- 소요 시간이 **살아있는 객체 수에 비례**한다.
- 살아있는 데이터가 많은 애플리케이션에서는 이 비용을 감당하기 어렵다.

그래서 실제 GC에는 매번 힙 전체를 뒤지지 않기 위한 최적화가 필요하다.

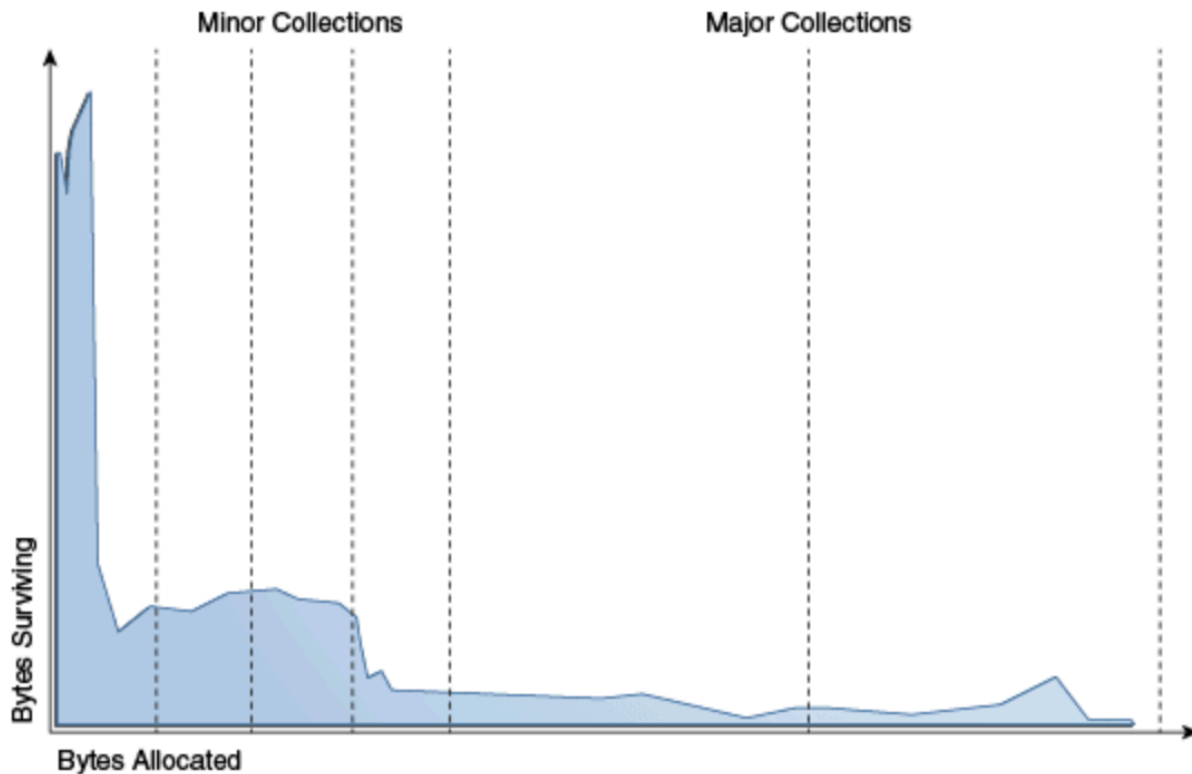
2. 약한 세대 가설

HotSpot의 GC들은(ZGC 제외) **세대별 수집(generational collection)** 기법을 사용한다. 이 기법은 애플리케이션에서 경험적으로 관찰되는 성질을 이용하는데, 그중 가장 중요한 것이 **약한 세대 가설(Weak Generational Hypothesis)** 이다.

대부분의 객체는 아주 짧은 시간만 생존한다.

Note: "ZGC 제외"는 JDK 17 기준이다. JDK 21부터 Generational ZGC가 도입되어, 이제는 ZGC도 세대별 수집을 지원한다.

Figure 3-1 Typical Distribution for Lifetimes of Objects



Description of "Figure 3-1 Typical Distribution for Lifetimes of Objects"

객체 수명 분포를 그려보면, 왼쪽의 날카로운 봉우리에 주목하게 된다. 할당되자마자 곧 죽는 객체들이다. 예를 들면 반복문 한 번 도는 동안만 살아있는 iterator 객체 같은 것이다.

어차피 대부분의 객체가 금방 죽는다면, 굳이 힙 전체를 매번 뒤질 필요가 없다. 젊은 객체들이 모인 구역만 자주 청소하면 훨씬 효율적이다. 이것이 세대별 수집의 핵심 아이디어다.

3. 세대 구조와 Minor / Major Collection

이 가설을 활용하기 위해, 메모리를 **세대(Generation)** 로 나누어 관리한다.

세대(Generation): 나이가 다른 객체들을 담는 메모리 풀.

각 세대가 가득 차면, 그 세대에 대해 GC가 일어난다.

대부분의 객체는 **Young Generation** 에 할당되고, 대부분 여기서 죽는다.

Young이 가득 차면 Minor Collection이 발생한다.

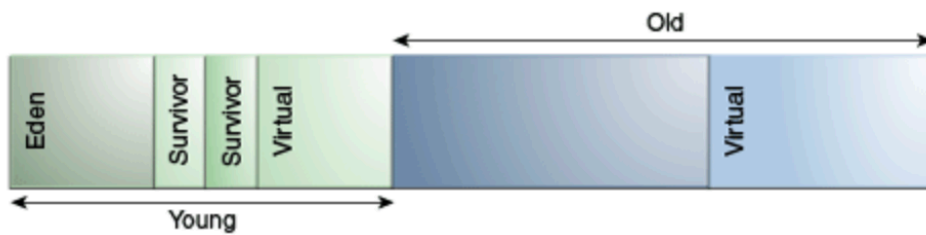
- 이때는 Young 세대만 수집하고 다른 세대는 건드리지 않는다.
- 수집 비용은 살아있는 객체 수에 비례한다. 죽은 객체로 가득 찬 Young은 빠르게 청소된다.
- Minor Collection마다 살아남은 객체의 일부는 **Old Generation** 으로 옮겨진다(승격, promotion).

Old마저 가득 차면 Major Collection(전체 힙 수집)이 발생한다.

- 관여하는 객체 수가 많아 Minor보다 오래 걸린다.

3.1 Young Generation 내부 구조

Figure 3-2 Default Arrangement of Generations in the Serial Collector



Description of "Figure 3-2 Default Arrangement of Generations in the Serial Collector"

Young Generation은 **Eden 하나 + Survivor 공간 2개** 로 구성된다.

- 대부분의 객체는 **Eden**에 처음 할당된다.
- 두 Survivor 공간 중 하나는 항상 비어 있다.
- GC가 일어나면, Eden과 (사용 중인) 다른 Survivor에서 살아남은 객체를 이 비어 있던 Survivor로 복사한다.
- GC가 끝나면 Eden과 원본 Survivor는 비워지고, 다음 GC 때 두 Survivor의 역할이 뒤바뀐다.
- 객체는 Survivor 사이를 오가며 복사되다가, 일정 횟수 이상 복사되거나 공간이 부족해지면 Old로 이동한다.

4. 세대 크기는 어떻게 정할까

세대 크기를 조절하면 아래 네 가지 지표가 서로 영향을 주고받는다.

- **Throughput**: 전체 시간 중 GC에 쓰지 **않은** 시간의 비율.
- **Latency**: 응답성. GC pause가 영향을 준다.
- **Footprint**: 프로세스의 워킹 셋(working set) 크기.
- **Promptness**: 객체가 죽은 시점부터 그 메모리가 실제로 다시 가용해지는 시점까지의 시간.

그렇다면 Young 영역 크기를 극단적으로 잡으면 어떻게 될까?

Q1. Young이 아주 크면?

- Eden이 자주 차지 않으므로 Minor Collection 빈도가 줄고, 객체가 Young에서 죽을 시간을 더 버니 Old로의 승격도 줄어든다 → throughput에 유리.
- 하지만 힙 크기가 고정이라면 Young이 클수록 Old가 작아진다 → 더 비싼 Major Collection이 자주 발생.
- 한 번의 Minor Collection이 다뤄야 할 영역도 커져 pause가 길어질 수 있다 → latency에 불리.

Q2. Young이 아주 작으면?

- Eden이 금방 차서 Minor Collection이 자주 발생한다 → throughput에 불리.
- 대신 각 pause 자체는 짧다.
- 객체가 충분히 나이 들기 전에 Old로 밀려나(조기 승격) Old를 빠르게 채울 수 있다.

결국 **세대 크기에 하나의 정답은 없다.** 최선은 애플리케이션의 메모리 사용 방식과 사용자 요구사항에 따라 달라진다.